CN-Stats-Lec08-Central Limit Theorem & Sampling Distributions & Hypothesis Testing

Central Limit Theorem & Sampling Distributions & Hypothesis Testing

🎯 Session Objectives:

- ✅ Define CLT and know when it applies across different data types.
- 🔁 Connect to simulations you've run and apply CLT in practice.
- 🧠 Why it matters: use CLT for inference/normal approximations in real analyses.
- 📊 Sampling distributions: build them, describe their shape, and see how n ↑ ⇒ shape ~ normal.
- 🧮 Compute sampling mean & standard error; understand SE = σ/√n and its link to SD under CLT.
- ✅ Understand and articulate the steps involved in hypothesis testing.
- ✅ Formulate null and alternate hypotheses for various scenarios.
- ✅ Explain the role of normal and standard normal distributions in hypothesis testing.
- ✅ Define and interpret p-values and significance levels (alpha) in the context of hypothesis testing.
- ✅ Construct appropriate null and alternate hypotheses based on different practical situations.

```python
# Coin Toss Probability Distribution
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random

coin_flips = [random.choice(['H','T']) for _ in range(100)]
print(coin_flips)
```

```
['T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'H', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'T',
'H', 'H', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'H', 'H', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'H',
'H', 'H', 'H', 'T', 'H', 'T', 'H', 'T', 'H', 'T', 'T', 'T', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'T', 'H', 'T',
'T', 'H', 'H', 'H', 'T', 'T', 'H', 'H', 'H', 'H', 'H', 'T', 'H', 'T', 'T', 'H', 'T', 'T', 'T', 'H', 'T', 'T',
'H', 'T', 'T', 'H', 'H', 'H', 'H', 'H', 'H', 'H', 'H', 'H']
```

```python
head_count = 0
tail_count = 0
for coin in coin_flips:
    if coin == 'H': # head
        head_count +=1
    else: # tail
        tail_count +=1

print(head_count)
print(tail_count)
```

```
50
50
```

```python
coin_flips = [random.choice(['H','T']) for _ in range(25)]
print(coin_flips)
```

```
['T', 'T', 'T', 'H', 'H', 'T', 'H', 'T', 'H', 'H', 'T', 'T', 'T', 'H', 'T', 'T', 'T', 'T', 'H', 'T', 'H', 'T',
'T', 'T', 'T']
```

```python
head_count = 0
tail_count = 0
for coin in coin_flips:
    if coin == 'H': # head
        head_count +=1
    else: # tail
        tail_count +=1

print(head_count)
print(tail_count)
```

```
8
17
```

```python
import pandas as pd
fraud_data = pd.read_csv("fraud_data.csv")
fraud_data
```

	trans_date_trans_time	merchant	category	amt	city	state	lat	long	city_pop	job
0	04-01-2019 0:58	"Stokes, Christiansen and Sipes"	grocery_net	14.37	Wales	AK	64.7556	-165.6723	145	"Administrator, education"
1	04-01-2019 15:06	Predovic Inc	shopping_net	966.11	Wales	AK	64.7556	-165.6723	145	"Administrator, education"
2	04-01-2019 22:37	Wisozk and Sons	misc_pos	49.61	Wales	AK	64.7556	-165.6723	145	"Administrator, education"
3	04-01-2019 23:06	Murray-Smitham	grocery_pos	295.26	Wales	AK	64.7556	-165.6723	145	"Administrator, education"

```python
fraud_data.shape
```

```
(14446, 15)
```

```python
trails = fraud_data.shape[0]
trails
```

```
14446
```

```python
# Fraud Data -> Population Mean
population_mean = fraud_data['amt'].mean() # size = 14446
print(f"Population_Mean : {population_mean}")

# Sample Mean [size = 1000]
sample_mean_1000 = fraud_data['amt'].sample(1000, random_state = 18).mean()
print(f"Sample_Mean for size 1000 : {sample_mean_1000}")

# Sample Mean [size = 5000]
sample_mean_5000 = fraud_data['amt'].sample(5000, random_state = 18).mean()
print(f"Sample_Mean for size 5000 : {sample_mean_5000}")

# Sample Mean [size = 10000]
sample_mean_10000 = fraud_data['amt'].sample(10000, random_state = 18).mean()
print(f"Sample_Mean for size 10000 : {sample_mean_10000}")
```

```
Population_Mean : 124.43007337671327
Sample_Mean for size 1000 : 123.42808
Sample_Mean for size 5000 : 120.88621600000002
Sample_Mean for size 10000 : 123.268
```

```python
# Finding the mean for different sample size [10,50,100,500,1000,5000]

# Fraud Data -> Population Mean
population_mean = fraud_data['amt'].mean() # size = 14446
print(f"Population_Mean : {population_mean}") # 124.4

import numpy as np
sample_size = np.array([10,50,100,500,1000,5000])

# Calculating the sample mean for different sample size
sample_mean = [fraud_data['amt'].sample(size, random_state = 55).mean() for size in sample_size]
print(sample_mean)
```

```
Population_Mean : 124.43007337671327
[111.144, 127.5, 119.66140000000001, 132.90465999999998, 130.25172, 122.842748]
```

```python
sample_mean = []
for size in sample_size:
    s_mean = fraud_data['amt'].sample(size, random_state = 55).mean()
    sample_mean.append(s_mean)

print(sample_mean)
```

```
[111.144, 127.5, 119.66140000000001, 132.90465999999998, 130.25172, 122.842748]
```

```python
# Sample Size VS Sample Mean
for size , mean in zip(sample_size , sample_mean):
    print(f"Sample_Size {size} : Sample Mean {mean}")
```

```
Sample_Size 10 : Sample Mean 111.144
Sample_Size 50 : Sample Mean 127.5
Sample_Size 100 : Sample Mean 119.66140000000001
Sample_Size 500 : Sample Mean 132.90465999999998
Sample_Size 1000 : Sample Mean 130.25172
Sample_Size 5000 : Sample Mean 122.842748
```

```python
np.mean(sample_mean)
```

```
124.05075466666666
```

```python
print(f"Population_Mean : {population_mean}") # 124.4
```

```
Population_Mean : 124.43007337671327
```

```python
# Finding the mean for different sample size [100,100,100,100,100,100,100]

# Fraud Data -> Population Mean
population_mean = fraud_data['amt'].mean() # size = 14446
print(f"Population_Mean : {population_mean}") # 124.4

import numpy as np
sample_size = np.array([100,100,100,100,100,100,100])

# Calculating the sample mean for different sample size
sample_mean = [fraud_data['amt'].sample(size).mean() for size in sample_size]
print(sample_mean)
```

```
Population_Mean : 124.43007337671327
[126.213, 129.3856, 145.1372, 101.7949, 104.0254, 90.98460000000001, 145.40770000000003]
```

```python
# Sample Size VS Sample Mean
for size , mean in zip(sample_size , sample_mean):
    print(f"Sample_Size {size} : Sample Mean {mean}")
```

```
Sample_Size 100 : Sample Mean 126.213
Sample_Size 100 : Sample Mean 129.3856
Sample_Size 100 : Sample Mean 145.1372
Sample_Size 100 : Sample Mean 101.7949
Sample_Size 100 : Sample Mean 104.0254
Sample_Size 100 : Sample Mean 90.98460000000001
Sample_Size 100 : Sample Mean 145.40770000000003
```

```python
np.mean(sample_mean)
```

```
120.4212
```

```python
print(f"Population_Mean : {population_mean}") # 124.4
```

```
Population_Mean : 124.43007337671327
```

```python
sample_size = np.array([100 for i in range(100)])
sample_size
```

```
array([100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,
       100, 100, 100, 100, 100, 100, 100, 100, 100])
```

```python
# Finding the mean for different sample size [100 , for 100 times]

# Fraud Data -> Population Mean
population_mean = fraud_data['amt'].mean() # size = 14446
print(f"Population_Mean : {population_mean}") # 124.4

import numpy as np
sample_size = np.array([100 for i in range(100)])

# Calculating the sample mean for different sample size
sample_mean = [fraud_data['amt'].sample(size).mean() for size in sample_size]
sample_mean = [f"{_mean:.2f}" for _mean in sample_mean]
sample_mean = list(map(float, sample_mean))
print(sample_mean)
```

```
Population_Mean : 124.43007337671327
[90.19, 163.95, 98.81, 149.64, 106.79, 118.29, 143.84, 119.16, 141.15, 122.06, 120.07, 158.76, 182.37, 134.43,
166.91, 120.23, 143.05, 145.78, 112.67, 137.34, 100.42, 128.14, 119.75, 117.56, 196.66, 137.29, 101.13, 168.5
9, 140.56, 110.79, 70.56, 127.21, 157.98, 144.87, 91.21, 192.84, 90.16, 144.32, 114.37, 92.34, 108.76, 132.05,
98.2, 122.69, 139.29, 81.59, 141.06, 104.83, 107.78, 116.26, 78.17, 112.18, 167.38, 82.42, 88.04, 121.86, 100.
12, 143.07, 148.47, 105.73, 118.54, 139.03, 117.47, 116.14, 134.18, 126.83, 123.61, 103.05, 122.66, 136.22, 8
2.91, 96.82, 136.83, 158.55, 129.18, 116.08, 149.14, 159.17, 138.0, 105.26, 98.61, 101.73, 97.47, 106.96, 131.
88, 166.23, 120.39, 138.46, 132.81, 133.12, 99.64, 71.51, 117.82, 113.73, 112.15, 96.49, 127.37, 89.27, 179.3
3, 126.89]
```

```python
np.mean(sample_mean)
```

```
123.93720000000002
```

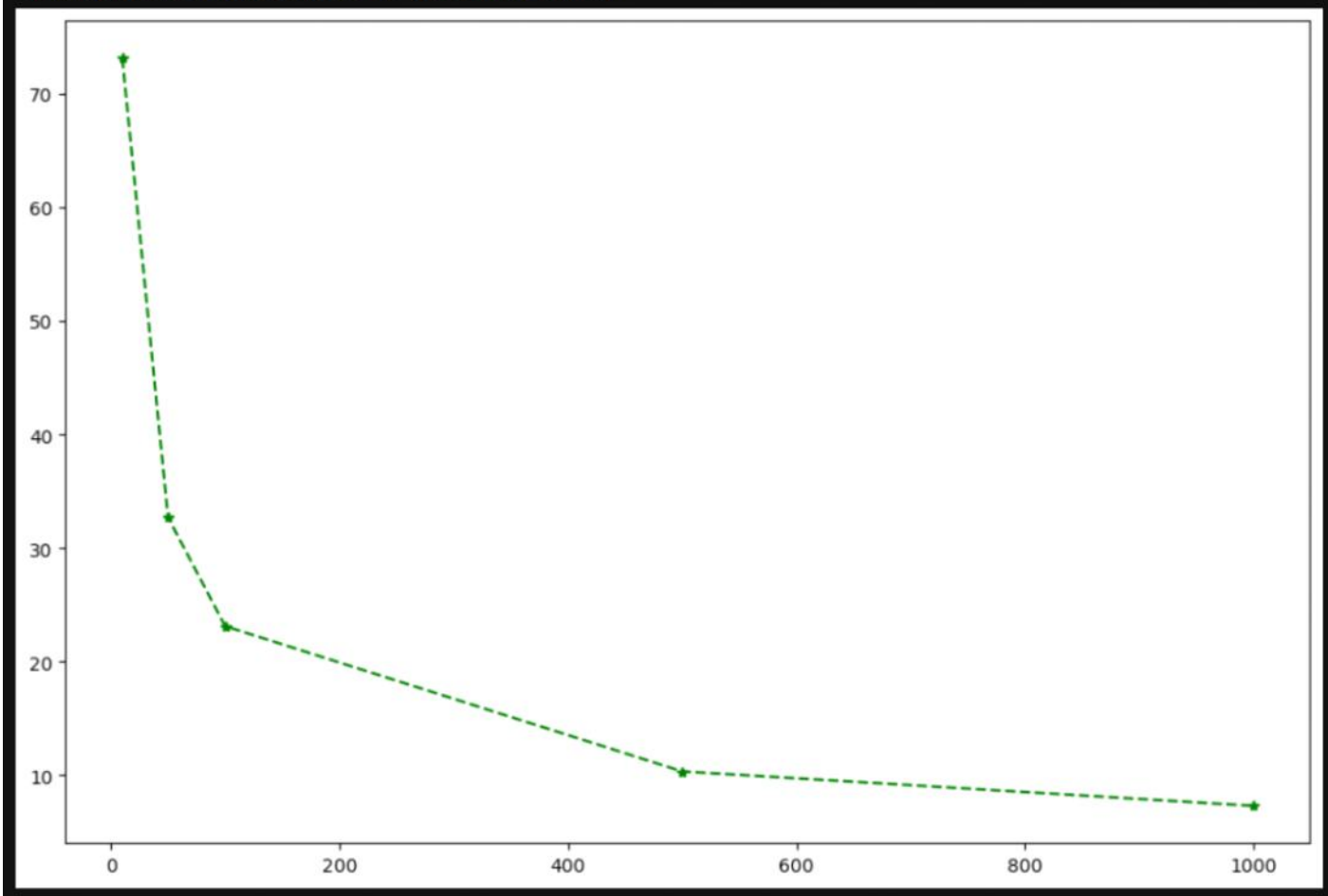
```python
print(f"Population_Mean : {population_mean}") # 124.4
```

```
Population_Mean : 124.43007337671327
```

```python
# Standard Error vs. Standard Deviation ['Visualization']
Population_std = fraud_data['amt'].std()
print(f"Population_STD : {Population_std}")

# Calculate SE = s / sqrt(n) # Math Module [np.sqrt]
sample_size = np.array([10,50,100,500,1000])
standard_error = [(Population_std / np.sqrt(size)) for size in sample_size]
standard_error
```

```
Population_STD : 231.35258718833137
[73.16011180878175,
 32.718196649184215,
 23.135258718833136,
 10.346402234471118,
 7.316011180878175]
```

```python
import matplotlib.pyplot as plt
plt.figure(figsize = (12,8))
plt.plot(sample_size , standard_error , marker = '*' , linestyle = '--' , color = 'green')
```

```
[<matplotlib.lines.Line2D at 0x257fb265640>]
```